JAVA PRACTICAL CODING WORKBOOK

Programming problems • Basic, Intermediate, Advanced

Topics included

Access modifiers, primitive data types, variables, operators, Strings, arrays, control flow, classes and objects, constructors and overloading, shallow/deep copying, input and output.

How to use this workbook

For every problem, plan first, write the code, compile and run it, then test normal and edge cases. Explain your solution in your own words.

BASIC — TYPES, VARIABLES, OUTPUT

1. Declare and print one variable of each primitive Java type.
Test / guidance: Include byte, short, int, long, float, double, char, and boolean.
Plan / notes: _____

2. Convert Celsius to Fahrenheit using variables and arithmetic operators.
Test / guidance: Test 0°C and 100°C.
Plan / notes: _____

3. Read two integers and print their sum, difference, product, quotient, and remainder.
Test / guidance: Test negative numbers and prevent division by zero.
Plan / notes: _____

4. Read marks in three subjects; print total and decimal average.
Test / guidance: Avoid integer division for the average.
Plan / notes: _____

5. Swap two integer variables using a temporary variable.
Test / guidance: Print values before and after swapping.
Plan / notes: _____

BASIC — STRINGS, CONDITIONS, LOOPS

6. Read first and last names and print the full name.
Test / guidance: Names may contain spaces; use line input where needed.
Plan / notes: _____

7. Read a number and print whether it is even or odd.
Test / guidance: Use the remainder operator.
Plan / notes: _____

8. Classify an integer as positive, negative, or zero using if/else.
Test / guidance: Test all three cases.
Plan / notes: _____

9. Print the multiplication table of a number from 1 to 10 using a for loop.
Test / guidance: Check both endpoints.
Plan / notes: _____

10. Use do-while to show a menu until the user selects Exit.
Test / guidance: The menu should appear at least once.
Plan / notes: _____

11. Create a switch-based calculator for +, -, *, and /.
Test / guidance: Handle invalid operators and division by zero.
Plan / notes: _____

INTERMEDIATE — ARRAYS

12. Read n integers into an array and print them in forward and reverse order.
Test / guidance: Test n = 1 and repeated values.
Plan / notes: _____

13. Find the minimum, maximum, and sum of an integer array.
Test / guidance: Define behavior for an empty array.
Plan / notes: _____

14. Count even, odd, positive, negative, and zero values in an array.
Test / guidance: Count zero consistently.
Plan / notes: _____

15. Search for a target value and print its first index or -1 if absent.
Test / guidance: Test beginning, end, duplicate, and missing targets.

Plan / notes: _____

16. Copy an array to a new array; modify the copy and prove the original is unchanged.
Test / guidance: Do not assign both variables to the same array reference.
Plan / notes: _____

17. Print all values greater than the array average.
Test / guidance: Calculate average using floating-point division.
Plan / notes: _____

INTERMEDIATE — CLASSES AND CONSTRUCTORS

18. Create a Student class with private name and marks fields, a constructor, and a display method.
Test / guidance: Create at least two Student objects.
Plan / notes: _____

19. Create a Rectangle class with no-argument and parameterized constructors; add area and perimeter methods.
Test / guidance: Use constructor overloading.
Plan / notes: _____

20. Create a Book class with overloaded constructors for title, title/author, and title/author/pages.
Test / guidance: Use this(...) chaining where appropriate.
Plan / notes: _____

21. Create a BankAccount with private balance, deposit, withdraw, and getBalance methods.
Test / guidance: Reject negative amounts and withdrawals beyond the balance.
Plan / notes: _____

22. Create a class with a private field and demonstrate access from inside and outside the class.
Test / guidance: Explain any compilation error.
Plan / notes: _____

INTERMEDIATE — SHALLOW AND DEEP COPYING

23. Create a Person class with a name and int[] scores; implement a shallow copy constructor and demonstrate shared-array behavior.
Test / guidance: Change a score through the copy and inspect the original.
Plan / notes: _____

24. Update the Person copy constructor to deep-copy the scores array.
Test / guidance: Changing the copied array must not change the original.
Plan / notes: _____

25. Create a class containing a mutable Address object; demonstrate shallow copy, then deep-copy Address as well.
Test / guidance: Show the effect of changing the copied address.
Plan / notes: _____

26. Write a copy method for a class with primitive, String, and array fields.
Test / guidance: Document which fields are shared and which are copied.
Plan / notes: _____

INTERMEDIATE — INPUT AND OUTPUT

27. Read a full name, age, and height using Scanner, then print a formatted summary.
Test / guidance: Handle the newline after nextInt() or nextDouble().
Plan / notes: _____

28. Read five values and print each with its index using both traditional and enhanced for loops.
Test / guidance: Explain why enhanced for does not directly provide the index.
Plan / notes: _____

29. Print a receipt with item, quantity, unit price, and total using printf().
Test / guidance: Display prices to two decimal places.
Plan / notes: _____

30. Keep asking for an integer until valid input is entered.
Test / guidance: Handle non-integer input without crashing.
Plan / notes: _____

ADVANCED — INTEGRATED PROJECTS

31. Build a Student Marks Manager using a Student class, private fields, overloaded constructors, an array of marks, and total/average/grade methods.
Test / guidance: Validate marks and test boundary scores.
Plan / notes: _____

32. Build a Library Book tracker with a Book class and an array of Book objects; support listing and title search.
Test / guidance: Compare titles using equals() or equalsIgnoreCase().
Plan / notes: _____

33. Build a menu-driven array utility with sum, average, min/max, search, reverse, and exit.
Test / guidance: Use switch and loops; handle invalid choices and empty arrays.
Plan / notes: _____

34. Create an Employee class with private fields and overloaded constructors; calculate a bonus and print a payslip.
Test / guidance: Validate salary and bonus rate.
Plan / notes: _____

35. Create a Contact class and a collection/array of contacts; implement copying and explain whether the copy is shallow or deep.
Test / guidance: Test copying, editing, searching, and empty input.
Plan / notes: _____

Completion checklist

■ Code compiles. ■ Readable names. ■ Input validated. ■ Normal cases tested. ■ Edge cases tested. ■ Output is clear.